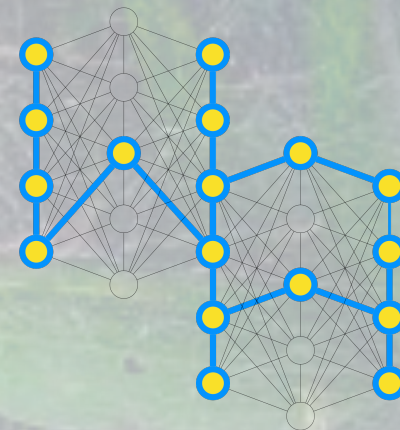


1222 • 2022
800
ANNI



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



Java Servlets and Access to the Database

Web Applications

Master Degree in Computer Engineering

Master Degree in Cybersecurity

Master Degree in ICT for Internet and Multimedia

Academic Year 2023/2024



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

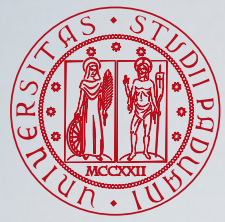
Nicola Ferro

Intelligent Interactive Information Access (IIIA) Hub

Department of Information Engineering

University of Padua



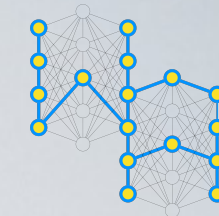
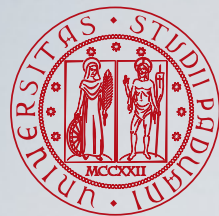


Outline

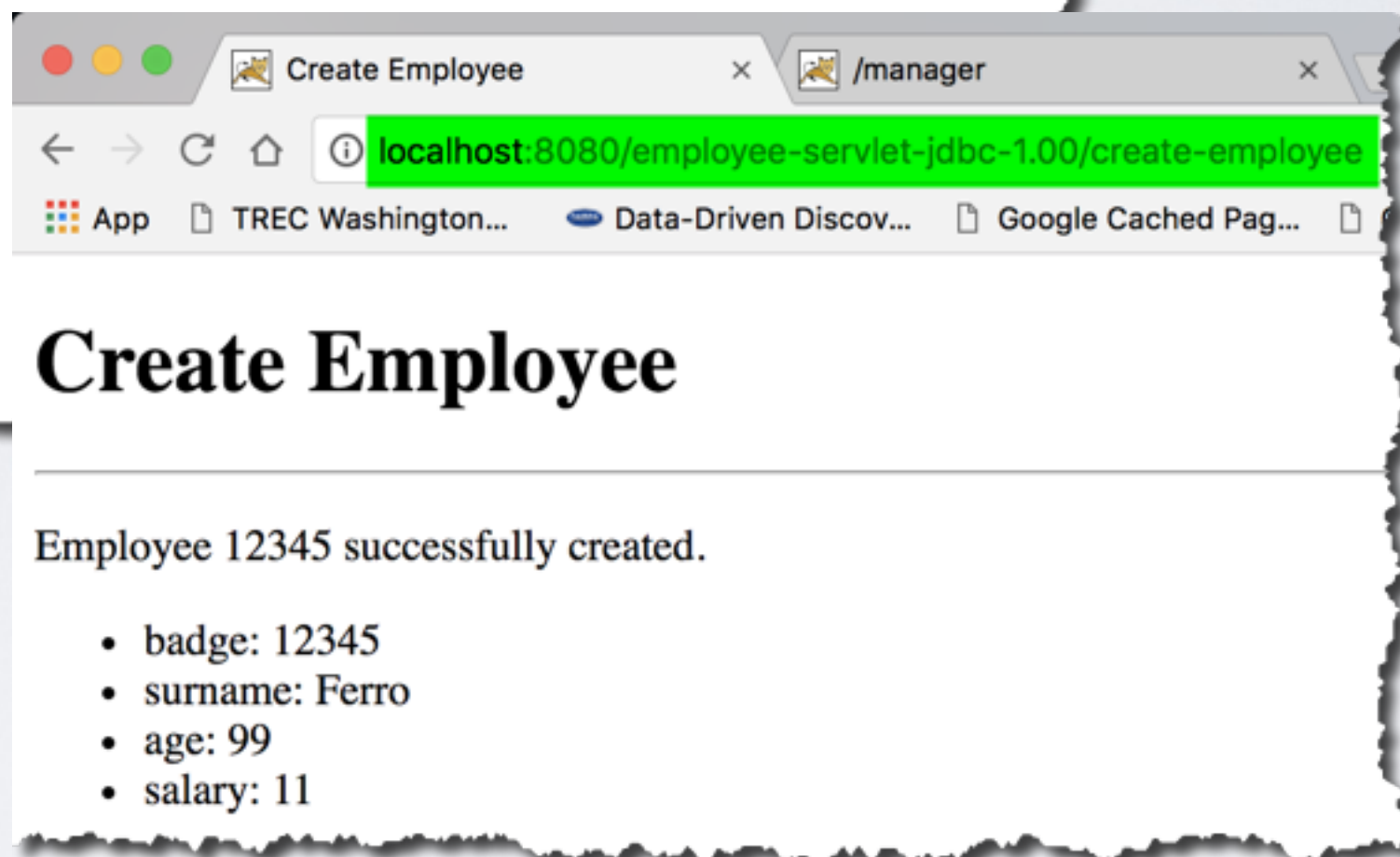
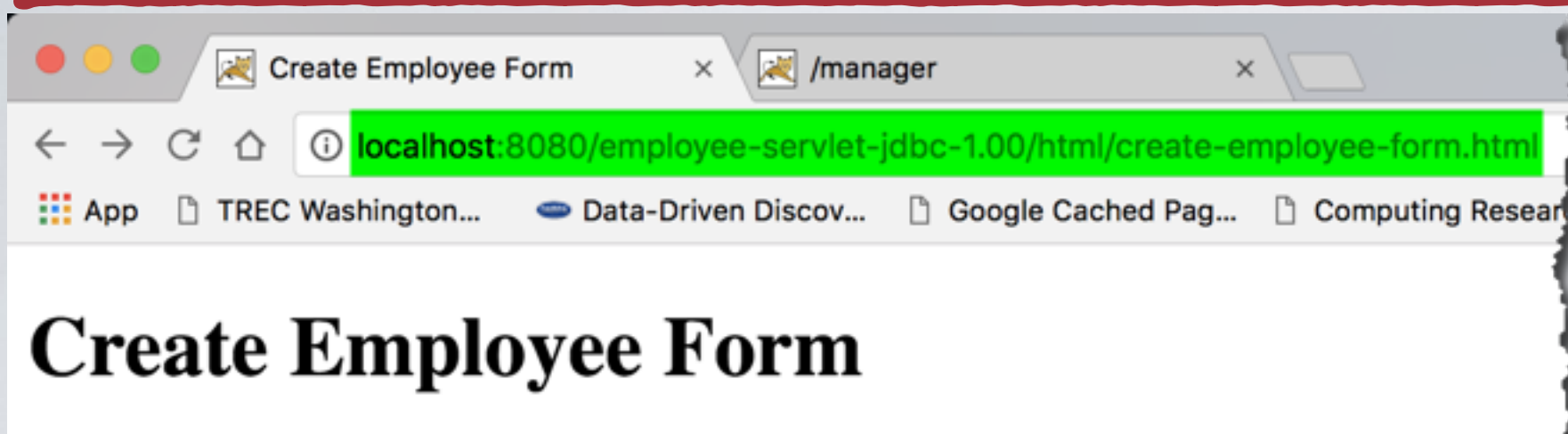


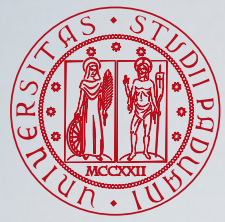
- Overall architecture of a full-stack application
- The resource (Java Beans) classes
- The Data Access Object (DAO) pattern
- Pool of connections
- SQL Injection

Overall Architecture

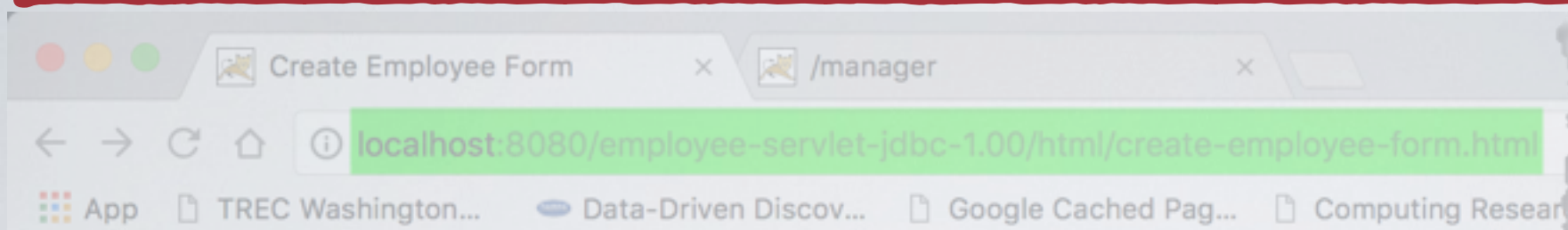


Application Functionalities: Create Employee





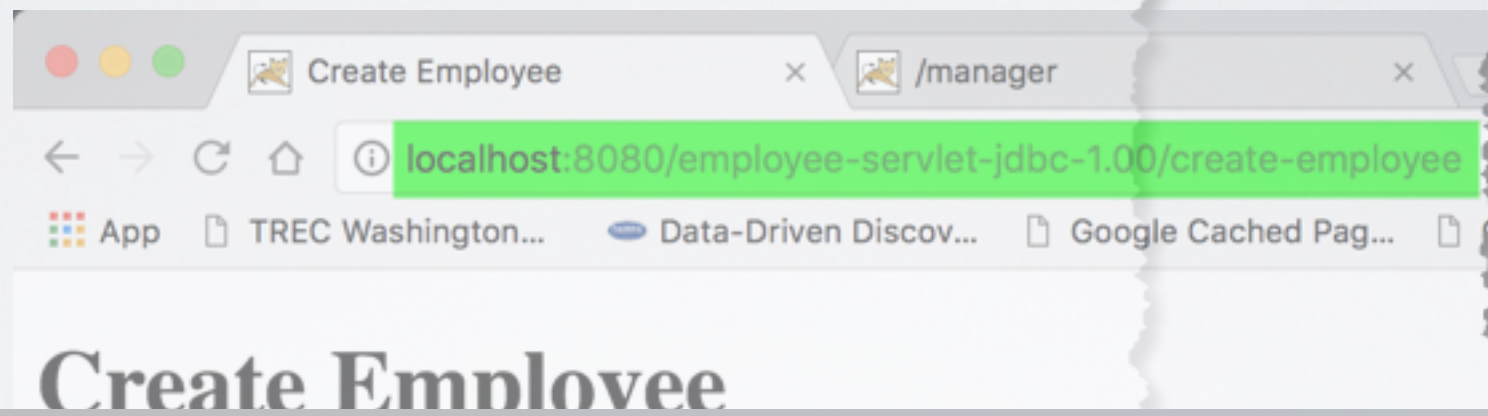
Application Functionalities: Create Employee



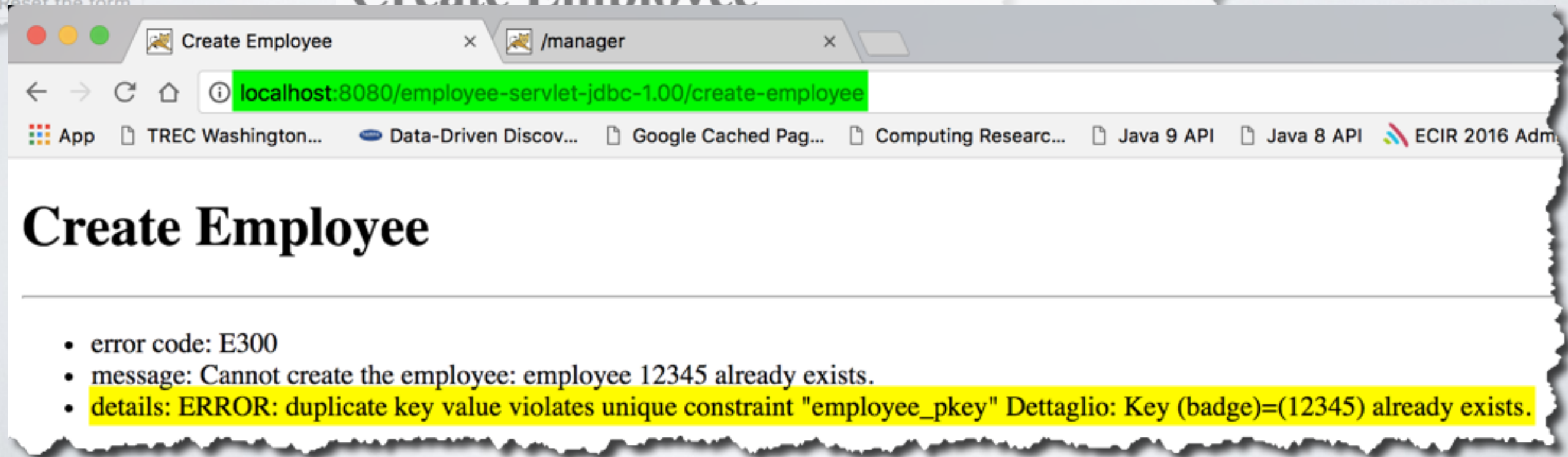
Create Employee Form

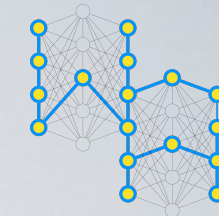
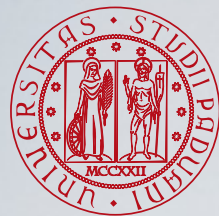
Badge: 12345
Surname: Ferro
Age: 99
Salary: 11

Submit
Reset the form

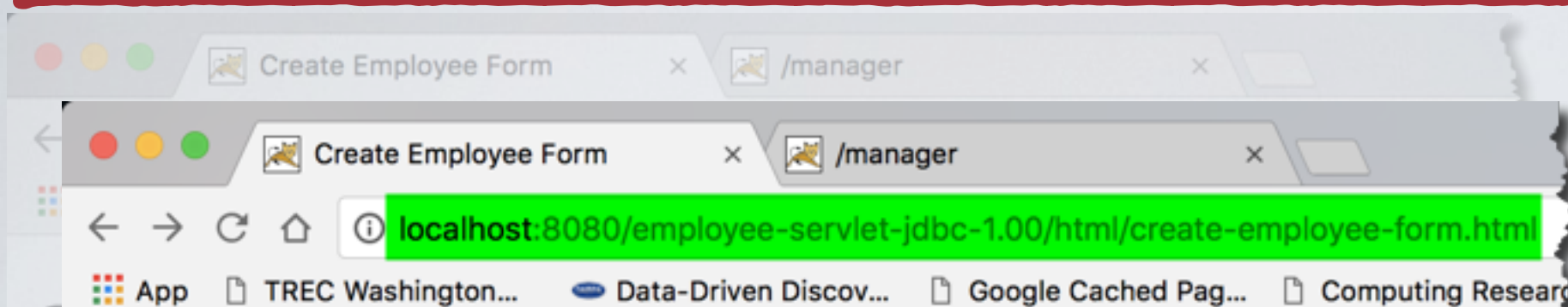


Create Employee





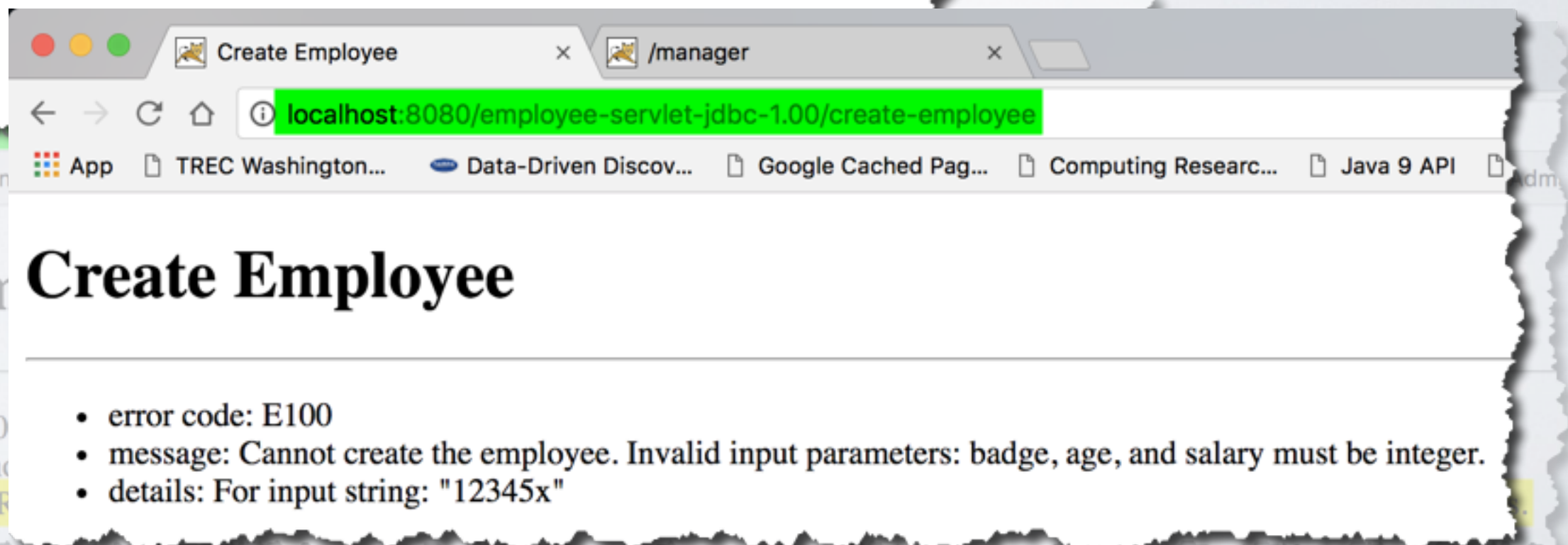
Application Functionalities: Create Employee



Create Employee Form

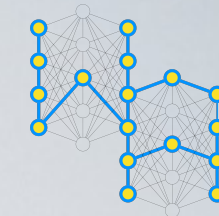
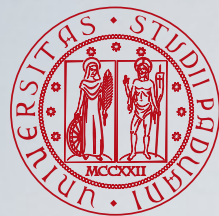
Badge: 12345x
Surname: Ferro
Age: 99
Salary: 11

Submit
Reset the form



Create Employee

- error code: E100
- message: Cannot create the employee. Invalid input parameters: badge, age, and salary must be integer.
- details: For input string: "12345x"



Application Functionalities: Search Employee

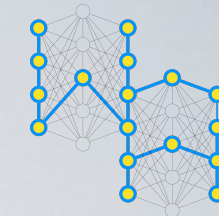
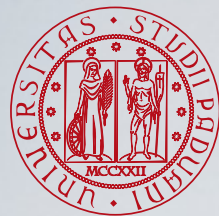
Search Employee Form

Salary:

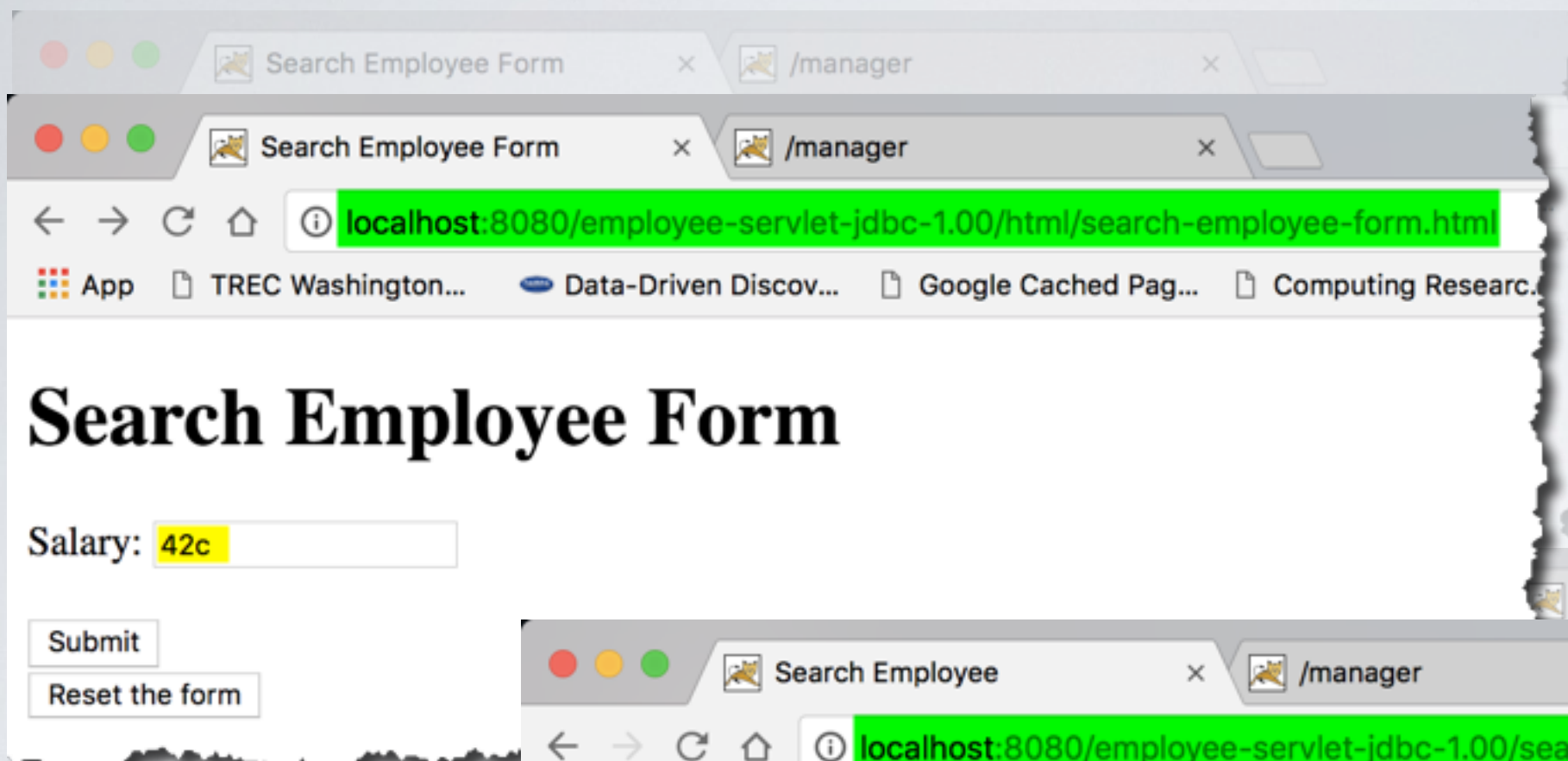
Search Employee

Employees successfully searched.

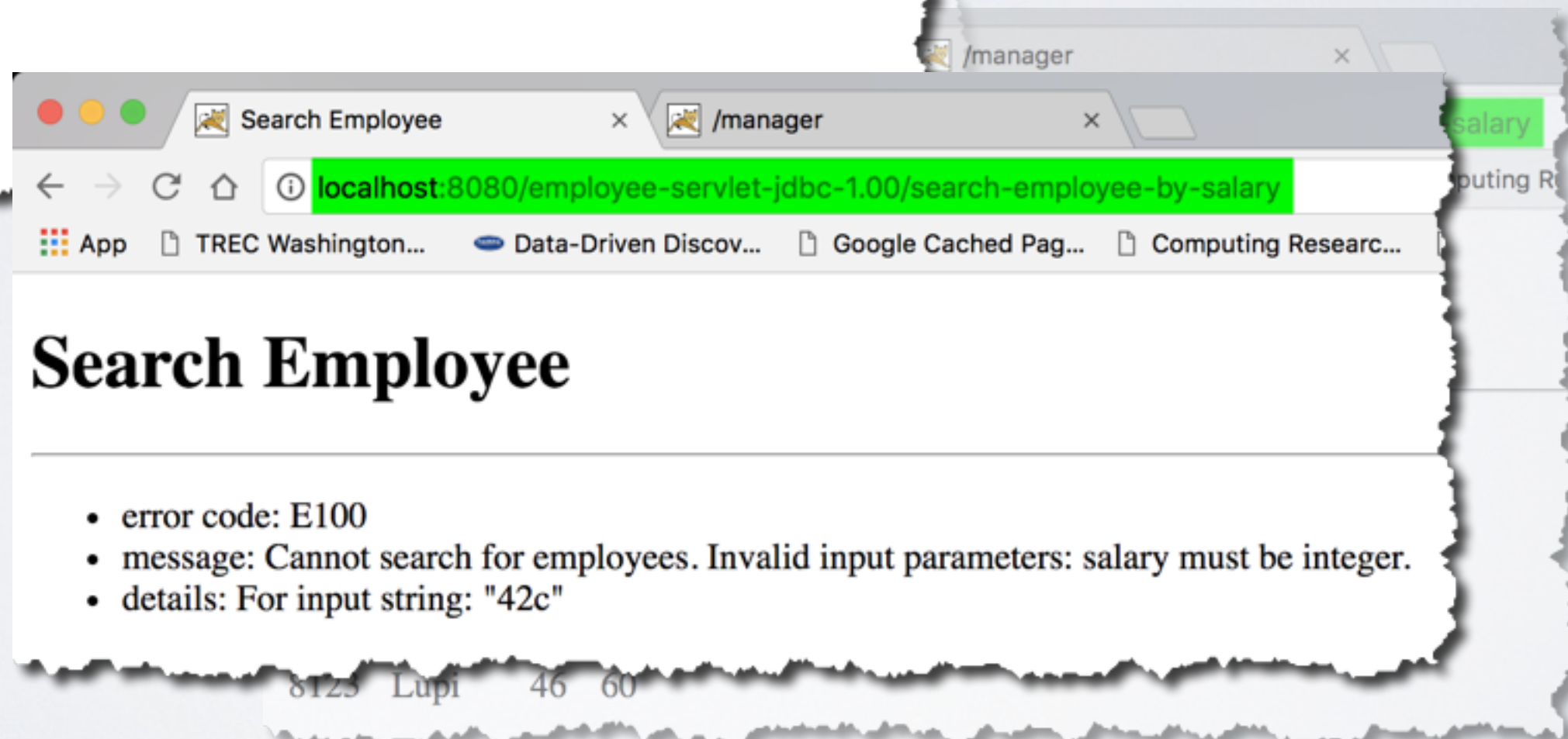
Badge	Surname	Age	Salary
7309	Rossi	34	45
4076	Mori	45	50
8123	Lupi	46	60



Application Functionalities: Search Employee



The screenshot shows a web browser window with two tabs: "Search Employee Form" and "/manager". The address bar displays "localhost:8080/employee-servlet-jdbc-1.00/html/search-employee-form.html". The page title is "Search Employee Form". Below the title, there is a "Salary:" label followed by a text input field containing "42c". At the bottom of the form, there are two buttons: "Submit" and "Reset the form".



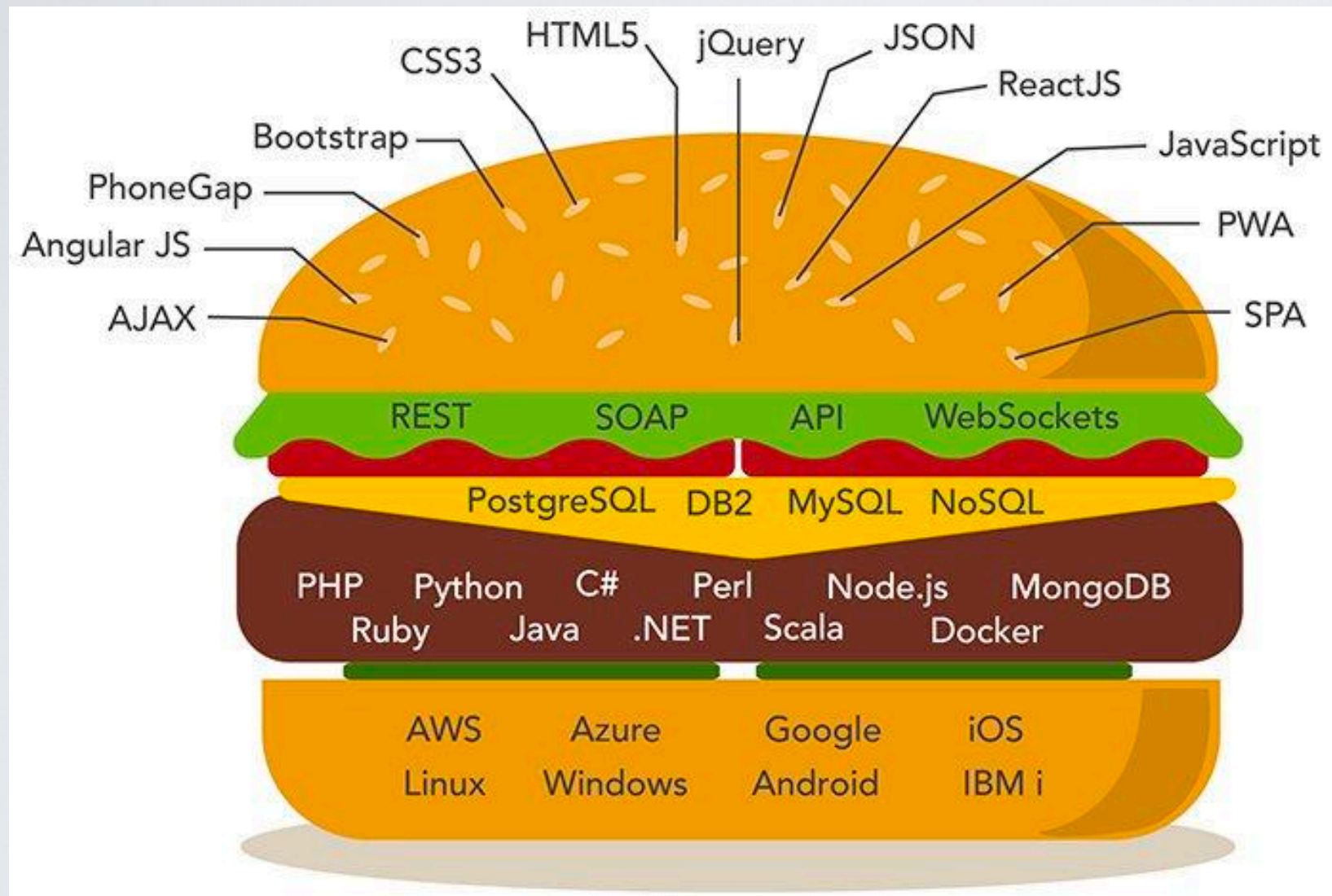
The screenshot shows a web browser window with two tabs: "Search Employee" and "/manager". The address bar displays "localhost:8080/employee-servlet-jdbc-1.00/search-employee-by-salary". The page title is "Search Employee". Below the title, there is a list of error details:

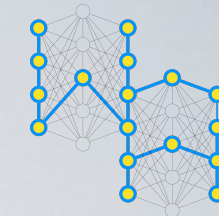
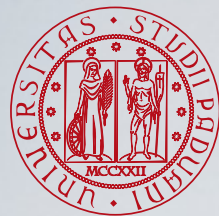
- error code: E100
- message: Cannot search for employees. Invalid input parameters: salary must be integer.
- details: For input string: "42c"

Below the error message, there is a table with the following data:

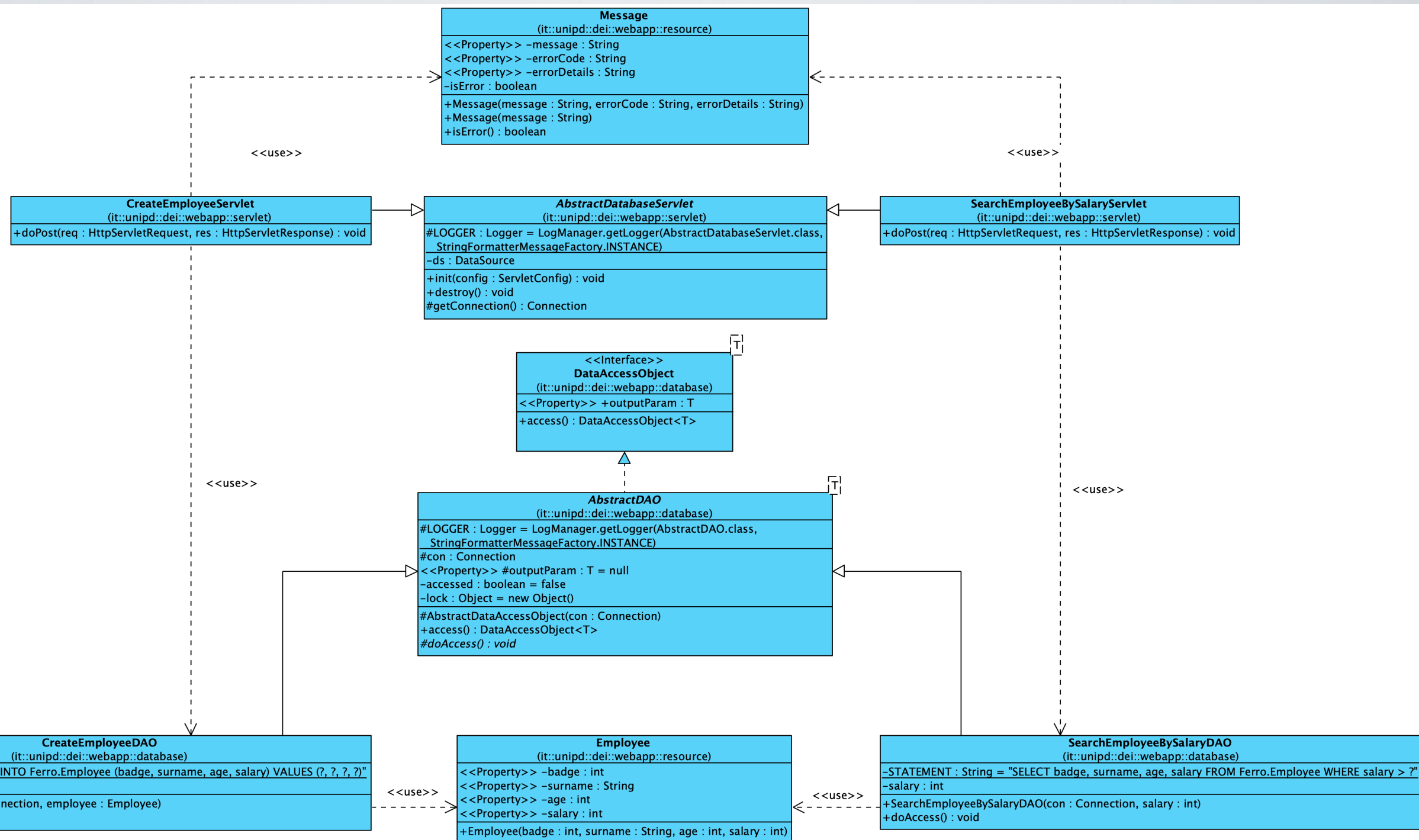
id	name	salary
8123	Lupi	46 60

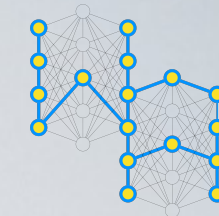
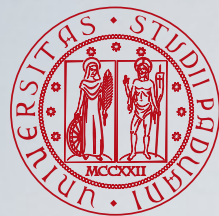
The Full-Stack



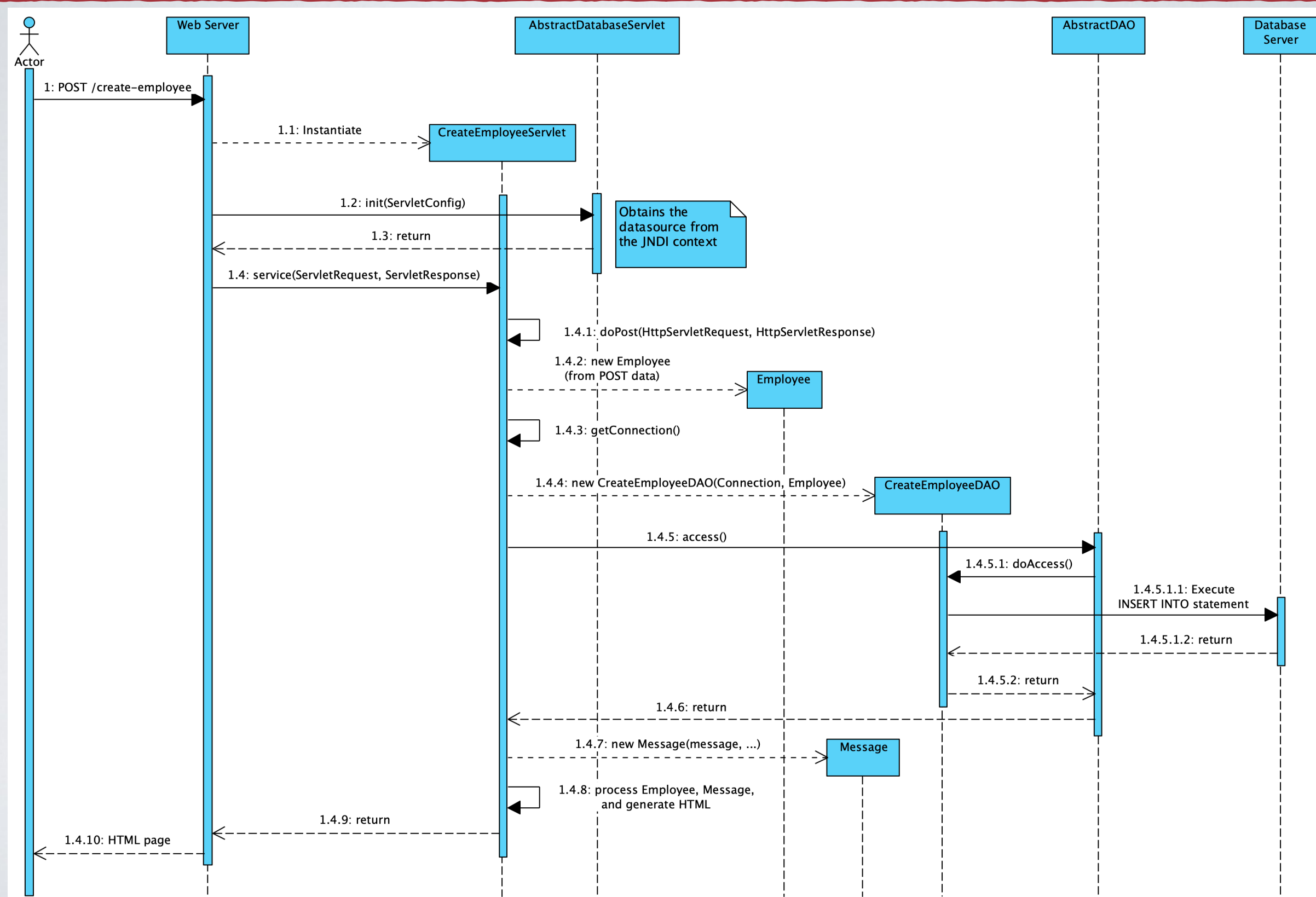


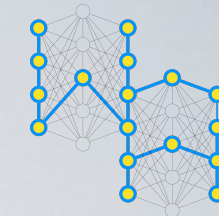
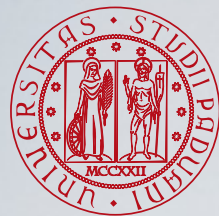
Employee Web Application Class Diagram



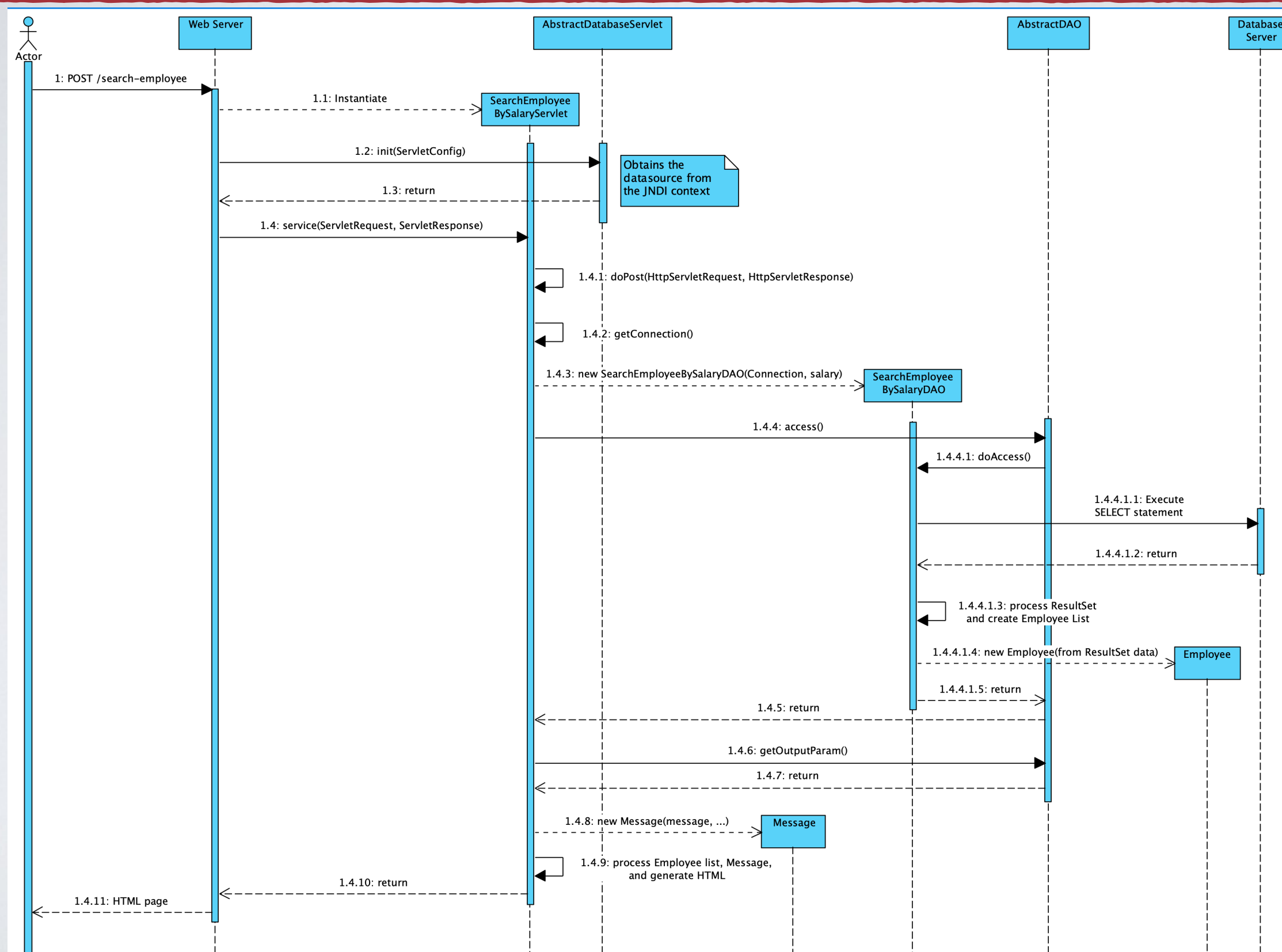


Create Employee: Sequence Diagram

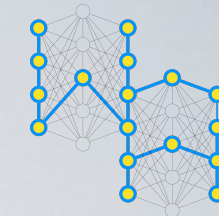
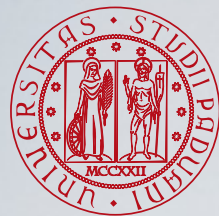




Search Employee: Sequence Diagram



The Data Logic Layer



The Employee Database

Employee	Manager
7309	5698
5998	5698
9553	4076
5698	4076
4076	8123

Manage

Employee

Badge	Surname	Age	Salary
7309	Rossi	34	45
5998	Bianchi	37	38
9553	Neri	42	35
5698	Bruni	43	42
4076	Mori	45	50
8123	Lupi	46	60

The Employee Class

```
package it.unipd.dei.webapp.resource;
```

```
public class Employee {
```

```
    private final int badge;
```

```
    private final String surname;
```

```
    private final int age;
```

```
    private final int salary;
```

```
    public Employee(final int badge, final String surname, final int age, final int salary) {  
        this.badge = badge;  
        this.surname = surname;  
        this.age = age;  
        this.salary = salary;  
    }
```

```
    public final int getBadge() {  
        return badge;  
    }
```

```
    public final String getSurname() {  
        return surname;  
    }
```

```
    public final int getAge() {  
        return age;  
    }
```

```
    public final int getSalary() {  
        return salary;  
    }
```

```
}
```

Fields for describing an Employee.

Using **final** fields prevents any concurrency issues (remember servlets are not thread-safe)

The Employee class can be subclassed but the accessor methods to the private fields are **final** to avoid breaking encapsulation if subclassing.

The Message Class

```
package it.unipd.dei.webapp.resource;
```

```
public class Message {
```

```
    private final String message;
```

```
    private final String errorCode;
```

```
    private final String errorDetails;
```

```
    private final boolean isError;
```

```
    public Message(final String message, final String errorCode, final String errorDetails) {
```

```
        this.message = message;
```

```
        this.errorCode = errorCode;
```

```
        this.errorDetails = errorDetails;
```

```
        this.isError = true;
```

```
    }
```

```
    public Message(final String message) {
```

```
        this.message = message;
```

```
        this.errorCode = null;
```

```
        this.errorDetails = null;
```

```
        this.isError = false;
```

```
    }
```

```
    public final String getMessage() {
```

```
        return message;
```

```
    }
```

```
    public final String getErrorCode() {
```

```
        return errorCode;
```

```
    }
```

```
    public final String getErrorDetails() {
```

```
        return errorDetails;
```

```
    }
```

```
    public final boolean isError() {
```

```
        return isError;
```

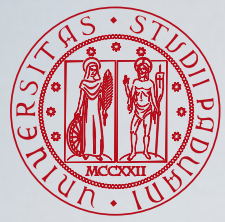
```
    }
```

```
}
```

The Message class represents both informative and error messages

This constructor is used for error messages

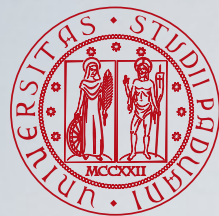
This constructor is used for informative messages



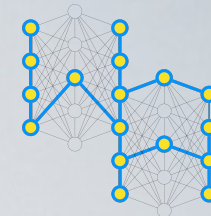
The Data Access Object (DAO) Interface

```
public interface DataAccessObject<T> {  
  
    /**  
     * Accesses the database.  
     *  
     * @return a reference to this {@code DataAccessObject} object.  
     *  
     * @throws SQLException if something goes wrong while accessing the database.  
     */  
    6 usages 1 implementation  Nicola Ferro  
    DataAccessObject<T> access() throws SQLException;  
  
    /**  
     * Retrieves any output parameters, after the access to the database.  
     *  
     * @return output parameter, or @code{null} if there is no output parameter.  
     */  
    2 usages 1 implementation  Nicola Ferro  
    T getOutputParam();  
}
```

- The Data Access Object (DAO) pattern abstracts and encapsulates all the logic need to access a data source, typically a relational database
 - <https://www.oracle.com/java/technologies/dataaccessobject.html>
- Each DAO is responsible for the persistence of a given resource, e.g. Employee in our case
- All the DAO implement a common interface and each of them provides an access operation, e.g. create Employee, read Employee, ...
 - the generic type T is the class of the resource the DAO is about, e.g. Employee
 - the access method performs the actual access to the datasource
 - the getOutputParam method is used to retrieve any output parameter resulting from the access to the datasource, if any
- The use of a common interface potentially allows for automating the DAO creation and use via reflection

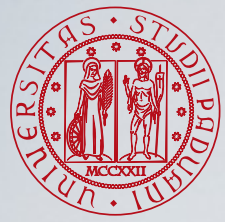


The AbstractDAO class



- The `AbstractDAO` class provides a base implementation of the `DataAccessObject` interface so that all the subclasses have a **uniform behavior** and focus just on implementing the **specific logic** for performing the requested data access operation
- The implementation of the `access()` method takes care of always closing the connection to the database and of rolling-back the transaction, if needed
- The `access()` method delegates the actual logic to perform the access to the datasource to its subclasses, via the abstract `doAccess()` method which has to be implemented by them
- DAO objects are one-shot and they are not expected to be re-used; in this respect, they would not need to be concerned with thread-safety.
 - However, the `AbstractDAO` class assumes the possibility of a mis-use of a DAO (or leakage) and manages, to a certain extent, concurrency issues via `lock` object and the `accessed` flag

```
▼ AbstractDAO
  (m) ? AbstractDAO(Connection)
  (m) ? access(): DataAccessObject<T> ↑DataAccessObject
  (m) ? getOutputParam(): T ↑DataAccessObject
  (m) ? doAccess(): void
  (f) ? LOGGER: Logger = LogManager.getLogger(...)
  (f) ? con: Connection
  (f) ? outputParam: T = null
  (f) ? accessed: boolean = false
  (f) ? lock: Object = new Object()
```



The CreateEmployeeDatabase Class

```
public final class CreateEmployeeDAO extends AbstractDAO {
```

There is no output parameter, so no need of a generic

```
private static final String STATEMENT = "INSERT INTO Ferro.Employee (badge, surname, age, salary) VALUES (?, ?, ?, ?)";
```

```
private final Employee employee;
```

```
public CreateEmployeeDAO(final Connection con, final Employee employee) {  
    super(con);
```

The SQL statement to be executed

```
    if (employee == null) {  
        LOGGER.error("The employee cannot be null.");  
        throw new NullPointerException("The employee cannot be null.");  
    }
```

```
    this.employee = employee;  
}
```

```
@Override  
protected final void doAccess() throws SQLException {
```

```
    PreparedStatement pstmt = null;
```

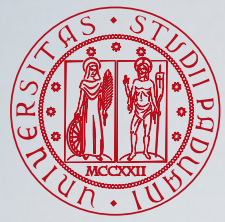
```
    try {  
        pstmt = con.prepareStatement(STATEMENT);  
        pstmt.setInt(1, employee.getBadge());  
        pstmt.setString(2, employee.getSurname());  
        pstmt.setInt(3, employee.getAge());  
        pstmt.setInt(4, employee.getSalary());
```

The class is not concerned with obtaining the connection to the database or the data about the employee, which are just passed from the classes calling it

```
        pstmt.execute();
```

```
        LOGGER.info("Employee %d successfully stored in the database.", employee.getBadge());  
    } finally {  
        if (pstmt != null) {  
            pstmt.close();  
        }  
    }  
}
```

The class just focuses on accessing the database, copies to/from application data structures, ensures to release the connection, and delegates the management of any exception to the super class (AbstractDAO)



The SearchEmployeeBySalaryDAO Class

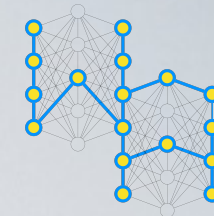
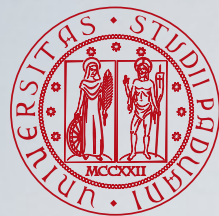
The type of the output parameter as a generic

The SQL statement to be executed

Processes the ResultSet, for each row, creates a new Employee object corresponding to that row, and appends it to the List of employees to be returned

Sets the output parameter that can be retrieved by calling `getOutputParameter()`

```
public final class SearchEmployeeBySalaryDAO extends AbstractDAO<List<Employee>> {  
  
    private static final String STATEMENT = "SELECT badge, surname, age, salary FROM Ferro.Employee WHERE salary > ?";  
  
    private final int salary;  
  
    public SearchEmployeeBySalaryDAO(final Connection con, final int salary) {  
        super(con);  
        this.salary = salary;  
    }  
  
    @Override  
    public final void doAccess() throws SQLException {  
  
        PreparedStatement pstmt = null;  
        ResultSet rs = null;  
  
        // the results of the search  
        final List<Employee> employees = new ArrayList<Employee>();  
  
        try {  
            pstmt = con.prepareStatement(STATEMENT);  
            pstmt.setInt(1, salary);  
  
            rs = pstmt.executeQuery();  
  
            while (rs.next()) {  
                employees.add(new Employee(rs.getInt("badge"), rs.getString("surname"), rs.getInt("age"),  
                    rs.getInt("salary")));  
            }  
  
            LOGGER.info("Employee(s) with salary above %d successfully listed.", salary);  
        } finally {  
            if (rs != null) {  
                rs.close();  
            }  
  
            if (pstmt != null) {  
                pstmt.close();  
            }  
        }  
  
        this.outputParam = employees;  
    }  
}
```

Pool of Database Connections via Tomcat: context.xml

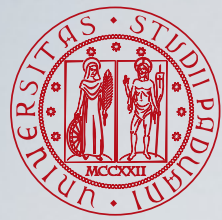
<Context>

```
<Resource name="jdbc/employee-ferro"
auth="Container"
type="javax.sql.DataSource"
factory="org.apache.tomcat.jdbc.pool.DataSourceFactory"
driverClassName="org.postgresql.Driver"
url="jdbc:postgresql://localhost:5432/esami"
username="ferro"
password="ferro"
testOnBorrow="true"
validationQuery="SELECT 1"
timeBetweenEvictionRunsMillis="30000"
maxActive="10"
minIdle="5"
maxWait="10000"
initialSize="2"
removeAbandonedTimeout="60"
removeAbandoned="true"
closeMethod="close"
/>
```

</Context>

<https://tomcat.apache.org/tomcat-10.1-doc/jdbc-pool.html>

<https://tomcat.apache.org/tomcat-10.1-doc/jndi-resources-howto.html>



Pool of Database Connections via Tomcat: context.xml

<Context>

<Resource name="jdbc/employee-ferro"

auth="Container"

type="javax.sql.DataSource"

factory="org.apache.tomcat.jdbc.pool.DataSourceFactory"

driverClassName="org.postgresql.Driver"

url="jdbc:postgresql://localhost:5432/esami"

username="ferro"

password="ferro"

testOnBorrow="true"

validationQuery="SELECT 1"

timeBetweenEvictionRunsMillis="30000"

maxActive="10"

minIdle="5"

maxWait="10000"

initialSize="2"

removeAbandonedTimeout="60"

removeAbandoned="true"

closeMethod="close"

/>

</Context>

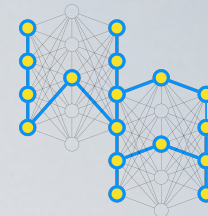
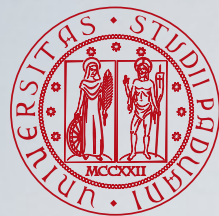
Java Naming and Directory Interface (JNDI) is a Java API for a directory service that allows Java software clients to **discover** and **look up** data and objects via a name.

You configure JNDI within Tomcat by putting a **context.xml** file within the **META-INF** folder where each **resource** elements configures a JNDI object which can be looked up afterwards. The **name** attribute assigns a unique name to that object.

You need to reference to the resource in the **web.xml** file in order to make it available for a Web application

<https://tomcat.apache.org/tomcat-10.1-doc/jdbc-pool.html>

<https://tomcat.apache.org/tomcat-10.1-doc/jndi-resources-howto.html>



Pool of Database Connections via Tomcat: context.xml

<Context>

<Resource name="jdbc/employee-ferro"

auth="Container"

type="javax.sql.DataSource"

factory="org.apache.tomcat.jdbc.pool.DataSourceFactory"

driverClassName="org.postgresql.Driver"

url="jdbc:postgresql://localhost:5432/esami"

username="ferro"

password="ferro"

testOnBorrow="true"

validationQuery="SELECT 1"

timeBetweenEvictionRunsMillis="30000"

maxActive="10"

minIdle="5"

maxWait="10000"

initialSize="2"

removeAbandonedTimeout="60"

removeAbandoned="true"

closeMethod="close"

Overall configuration of the connections:

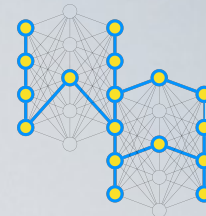
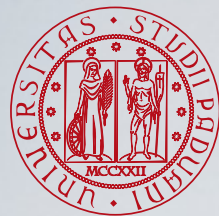
- **auth="Container"** means that Tomcat will perform the authentication via the provided parameters
- **type** specifies the type of resource, i.e. a JDBC connection pool
- **factory** specifies the class which will create instances of the pool, in our case we use the Tomcat connection pool implementation
- **driverClassName** the JDBC driver to be user
- **url, username, password** are the connection parameters

/>

</Context>

<https://tomcat.apache.org/tomcat-10.1-doc/jdbc-pool.html>

<https://tomcat.apache.org/tomcat-10.1-doc/jndi-resources-howto.html>



Pool of Database Connections via Tomcat: context.xml

<Context>

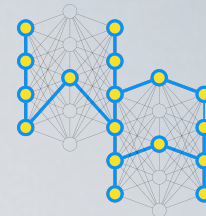
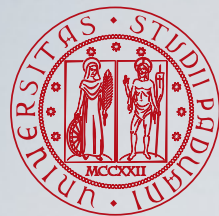
```
<Resource name="jdbc/employee-ferro"
  auth="Container"
  type="javax.sql.DataSource"
  factory="org.apache.tomcat.jdbc.pool.DataSourceFactory"
  driverClassName="org.postgresql.Driver"
  url="jdbc:postgresql://localhost:5432/esami"
  username="ferro"
  password="ferro"
  testOnBorrow="true"
  validationQuery="SELECT 1"
  timeBetweenEvictionRunsMillis="30000"
  maxActive="10"
  minIdle="5"
  maxWait="10000"
  initialSize="2"
  removeAbandonedTimeout="60"
  removeAbandoned="true"
  closeMethod="close"
/>
```

Advanced configuration parameters for the pool management, e.g. maximum and minimum number of connections, how to validate connections, and so on...

</Context>

<https://tomcat.apache.org/tomcat-10.1-doc/jdbc-pool.html>

<https://tomcat.apache.org/tomcat-10.1-doc/jndi-resources-howto.html>



Pool of Database Connections via Tomcat: context.xml

<Context>

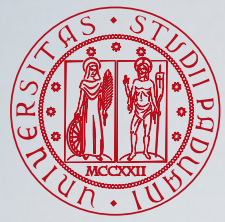
```
<Resource name="jdbc/employee-ferro"
auth="Container"
type="javax.sql.DataSource"
factory="org.apache.tomcat.jdbc.pool.DataSourceFactory"
driverClassName="org.postgresql.Driver"
url="jdbc:postgresql://localhost:5432/esam1"
username="ferro"
password="ferro"
testOnBorrow="true"
validationQuery="SELECT 1"
timeBetweenEvictionRunsMillis=50000
maxActive="10"
minIdle="5"
maxWait="10000"
initialSize="2"
removeAbandonedTimeout="60"
removeAbandoned="true"
closeMethod="close"
/>
```

The method to call on a singleton resource when it is no longer required. This is intended to speed up clean-up of resources that would otherwise happen as part of garbage collection.

Avoid the pool to run out of connection!

</Context>

<https://tomcat.apache.org/tomcat-10.1-doc/jdbc-pool.html>
<https://tomcat.apache.org/tomcat-10.1-doc/jndi-resources-howto.html>



The web.xml Configuration File



```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<web-app id="hello-world-webapp" version="4.0" xmlns="http://xmlns.jcp.org/xml/ns/javaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://java.sun.com/xml/ns/javaee http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd">
```

```
  <display-name>Employee Servlet JDBC</display-name>
  <description>Example servlet-based application accessing a DBMS via JDBC.</description>
```

```
  <servlet>
    <servlet-name>SearchEmployeeBySalary</servlet-name>
    <servlet-class>it.unipd.dei.webapp.servlet.SearchEmployeeBySalaryServlet</servlet-class>
  </servlet>
```

```
  <servlet>
    <servlet-name>CreateEmployee</servlet-name>
    <servlet-class>it.unipd.dei.webapp.servlet.CreateEmployeeServlet</servlet-class>
  </servlet>
```

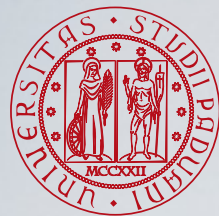
```
  <servlet-mapping>
    <servlet-name>SearchEmployeeBySalary</servlet-name>
    <url-pattern>/search-employee-by-salary</url-pattern>
  </servlet-mapping>
```

```
  <servlet-mapping>
    <servlet-name>CreateEmployee</servlet-name>
    <url-pattern>/create-employee</url-pattern>
  </servlet-mapping>
```

```
  <resource-ref>
    <description>Connection pool to the database</description>
    <res-ref-name>jdbc/employee-ferro</res-ref-name>
    <res-type>javax.sql.DataSource</res-type>
    <res-auth>Container</res-auth>
  </resource-ref>
```

```
</web-app>
```

Reference to the JDBC pool to make it available to the Web Application



Project Object Model (POM)



```
<!-- process resources before compilation and packaging -->
<resources>
```

```
  <!-- copy HTML files to the target directory -->
  <resource>
    <targetPath>${basedir}/target/${project.artifactId}-${project.version}/html</targetPath>
    <directory>${basedir}/src/main/webapp/html</directory>
    <includes>
      <include>**/*.*</include>
    </includes>
  </resource>
```

```
  <!-- copy configuration files to the target directory -->
  <resource>
    <targetPath>${basedir}/target/${project.artifactId}-${project.version}/META-INF</targetPath>
    <directory>${basedir}/src/main/webapp/META-INF</directory>
    <includes>
      <include>**/*.*</include>
    </includes>
  </resource>
```

```
</resources>
</build>
```

```
<!-- Dependencies -->
```

```
<dependencies>
  <dependency>
    <groupId>javax.servlet</groupId>
    <artifactId>javax.servlet-api</artifactId>
    <version>4.0.0</version>
    <scope>provided</scope>
  </dependency>
```

```
  <dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <version>42.2.2</version>
  </dependency>
```

```
  <dependency>
    <groupId>org.apache.tomcat</groupId>
    <artifactId>tomcat-jdbc</artifactId>
    <version>9.0.7</version>
    <scope>provided</scope>
  </dependency>
```

```
</dependencies>
```

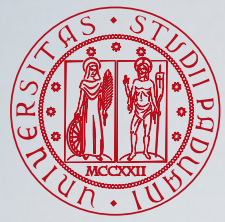
```
</project>
```

Copies the `context.xml` configuration file.

Adds the dependencies on the PostgreSQL JDBC driver and the Tomcat connection pool.

Note that the `scope` of the Tomcat connection pool is `provided` since it is already available in the deployment environment on Tomcat.

The Business&Interface Logic Layers



The Create and Search Employee Forms

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>Create Employee Form</title>
  </head>

  <body>
    <h1>Create Employee Form</h1>

    <form method="POST" action="../create-employee">
      <label for="badge">Badge:</label>
      <input name="badge" type="text"/><br/>

      <label for="surname">Surname:</label>
      <input name="surname" type="text"/><br/>

      <label for="age">Age:</label>
      <input name="age" type="text"/><br/>

      <label for="salary">Salary:</label>
      <input name="salary" type="text"/><br/><br/>

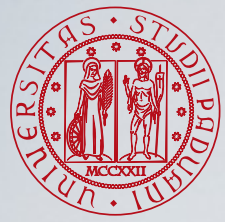
      <button type="submit">Submit</button><br/>
      <button type="reset">Reset the form</button>
    </form>
  </body>
</html>
```

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>Search Employee Form</title>
  </head>

  <body>
    <h1>Search Employee Form</h1>

    <form method="POST" action="../search-employee-by-salary">
      <label for="salary">Salary:</label>
      <input name="salary" type="text"/><br/><br/>

      <button type="submit">Submit</button><br/>
      <button type="reset">Reset the form</button>
    </form>
  </body>
</html>
```



The AbstractDatabaseServlet Class



```
public abstract class AbstractDatabaseServlet extends HttpServlet {
```

```
    protected static final Logger LOGGER = LogManager.getLogger(AbstractDatabaseServlet.class,
        StringFormatterMessageFactory.INSTANCE);
```

```
    private DataSource ds;
```

```
    public void init(ServletConfig config) throws ServletException {
```

```
        // the JNDI lookup context
        InitialContext cxt;
```

```
        try {
```

```
            cxt = new InitialContext();
            ds = (DataSource) cxt.lookup("java:/comp/env/jdbc/employee-ferro");
```

```
            LOGGER.info("Connection pool to the database pool successfully acquired.");
        } catch (NamingException e) {
            ds = null;
```

```
            LOGGER.error("Unable to acquire the connection pool to the database.", e);
```

```
            throw new ServletException("Unable to acquire the connection pool to the database", e);
```

```
        }
```

```
    }
```

```
    public void destroy() {
```

```
        ds = null;
```

```
        LOGGER.info("Connection pool to the database pool successfully released.");
```

```
    }
```

```
    protected final Connection getConnection() throws SQLException {
```

```
        try {
```

```
            return ds.getConnection();
```

```
        } catch (final SQLException e) {
```

```
            LOGGER.error("Unable to acquire the connection from the pool.", e);
```

```
            throw e;
```

```
        }
```

```
    }
```

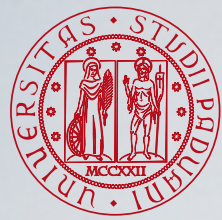
```
}
```

Use the `init` servlet lifecycle method to lookup the connection pool from JNDI

The `InitialContext` is the JNDI directory to look up (not to be confused with `ServletContext`) by using the name assigned to the JDBC pool in the `context.xml` file. Any look up problem will throw an exception.

The `DataSource` field cannot be `final`, since it is assigned outside the constructor.

Use the `DataSource` to return a connection to the database for a subclass



The CreateEmployeeServlet Class



```
public final class CreateEmployeeServlet extends AbstractDatabaseServlet {  
    public void doPost(HttpServletRequest req, HttpServletResponse res) throws IOException {
```

```
        LogContext.setIPAddress(req.getRemoteAddr());  
        LogContext.setAction(Actions.CREATE_EMPLOYEE);
```

```
        // request parameters  
        int badge = -1;  
        String surname = null;  
        int age = -1;  
        int salary = -1;
```

```
        // model  
        Employee e = null;  
        Message m = null;
```

```
        try {
```

```
            // retrieves the request parameters  
            badge = Integer.parseInt(req.getParameter("badge"));  
            surname = req.getParameter("surname");  
            age = Integer.parseInt(req.getParameter("age"));  
            salary = Integer.parseInt(req.getParameter("salary"));
```

```
            // set the badge of the employee as the resource in the log context at this point we know it is a valid integer  
            LogContext.setResource(req.getParameter("badge"));
```

```
            // creates a new employee from the request parameters  
            e = new Employee(badge, surname, age, salary);
```

```
            // creates a new object for accessing the database and stores the employee  
            new CreateEmployeeDAO(getConnection(), e).access();
```

```
            m = new Message(String.format("Employee %d successfully created.", badge));
```

```
            LOGGER.info("Employee %d successfully created in the database.", badge);
```

```
        } catch (NumberFormatException ex) {  
            m = new Message(  
                "Cannot create the employee. Invalid input parameters: badge, age, and salary must be integer.", "E100", ex.getMessage());  
            LOGGER.error("Cannot create the employee. Invalid input parameters: badge, age, and salary must be integer.", ex);  
        } catch (SQLException ex) {  
            if (ex.getSQLState().equals("23505")) {  
                m = new Message(String.format("Cannot create the employee: employee %d already exists.", badge), "E300", ex.getMessage());  
                LOGGER.error(new StringFormattedMessage("Cannot create the employee: employee %d already exists.", badge), ex);  
            } else {  
                m = new Message("Cannot create the employee: unexpected error while accessing the database.", "E200", ex.getMessage());  
                LOGGER.error("Cannot create the employee: unexpected error while accessing the database.", ex);  
            }  
        }  
    }
```

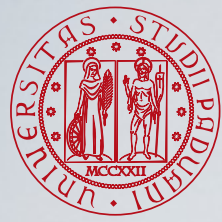
Retrieves request parameters and creates the corresponding Employee object

Retrieves the connection from the superclass and uses the helper DAO to access the database

```
        try {  
            // set the MIME media type of the response  
            res.setContentType("text/html; charset=utf-8");  
            // get a stream to write the response  
            PrintWriter out = res.getWriter();  
            // write the HTML page  
            out.printf("<!DOCTYPE html>%n");  
            out.printf("<html lang=\"%en\">%n");  
            out.printf("<head>%n");  
            out.printf("<meta charset=\"%utf-8\">%n");  
            out.printf("<title>Create Employee</title>%n");  
            out.printf("</head>%n");  
            out.printf("<body>%n");  
            out.printf("<h1>Create Employee</h1>%n");  
            out.printf("<hr/>%n");  
            if (m.isError()) {  
                out.printf("<ul>%n");  
                out.printf("<li>error code: %s</li>%n", m.getErrorCode());  
                out.printf("<li>message: %s</li>%n", m.getMessage());  
                out.printf("<li>details: %s</li>%n", m.getErrorDetails());  
                out.printf("</ul>%n");  
            } else {  
                out.printf("<p>%s</p>%n", m.getMessage());  
                out.printf("<ul>%n");  
                out.printf("<li>badge: %s</li>%n", e.getBadge());  
                out.printf("<li>surname: %s</li>%n", e.getSurname());  
                out.printf("<li>age: %s</li>%n", e.getAge());  
                out.printf("<li>salary: %s</li>%n", e.getSalary());  
                out.printf("</ul>%n");  
            }  
            out.printf("</body>%n");  
            out.printf("</html>%n");  
            // flush the output stream buffer  
            out.flush();  
            // close the output stream  
            out.close();  
        } catch (IOException ex) {  
            LOGGER.error(new StringFormattedMessage("Unable to send response when creating employee %d.", badge), ex);  
            throw ex;  
        } finally {  
            LogContext.removeIPAddress();  
            LogContext.removeAction();  
            LogContext.removeResource();  
        }  
    }
```

Writes out the HTML page

Manages error/success conditions and creates the corresponding Message



The SearchEmployeeBySalaryServlet Class

```
public final class SearchEmployeeBySalaryServlet extends AbstractDatabaseServlet {  
    public void doPost(HttpServletRequest req, HttpServletResponse res) throws IOException {
```

```
        LogContext.setIPAddress(req.getRemoteAddr());  
        LogContext.setAction(Actions.SEARCH_EMPLOYEE_BY_SALARY);
```

```
        // request parameter  
        int salary = -1;
```

```
        // model  
        List<Employee> el = null;  
        Message m = null;
```

```
        try {
```

```
            // retrieves the request parameter  
            salary = Integer.parseInt(req.getParameter("salary"));
```

```
            // creates a new object for accessing the database and searching the employees  
            el = new SearchEmployeeBySalaryDAO(getConnection(), salary).access().getOutputParam();
```

```
            m = new Message("Employees successfully searched.");
```

```
            LOGGER.info("Employees successfully searched by salary %d.", salary);
```

```
        } catch (NumberFormatException ex) {  
            m = new Message("Cannot search for employees. Invalid input parameters: salary must be integer.", "E100", ex.getMessage());
```

```
            LOGGER.error("Cannot search for employees. Invalid input parameters: salary must be integer.", ex);
```

```
        } catch (SQLException ex) {  
            m = new Message("Cannot search for employees: unexpected error while accessing the database.", "E200", ex.getMessage());
```

```
            LOGGER.error("Cannot search for employees: unexpected error while accessing the database.", ex);
```

```
        }
```

Retrieves the connection to the database, uses the helper DAO to access the database, and `getOutputParam()` to retrieve the list of `Employee` objects

```
        try {  
            // set the MIME media type of the response  
            res.setContentType("text/html; charset=utf-8");  
  
            // get a stream to write the response  
            PrintWriter out = res.getWriter();  
  
            // write the HTML page  
            out.printf("<!DOCTYPE html>%n");  
  
            out.printf("<html lang=\"%en\">%n");  
            out.printf("<head>%n");  
            out.printf("<meta charset=\"%utf-8\">%n");  
            out.printf("<title>Search Employee</title>%n");  
            out.printf("</head>%n");  
  
            out.printf("<body>%n");  
            out.printf("<h1>Search Employee</h1>%n");  
            out.printf("<hr/>%n");  
  
            if (m.isError()) {  
                out.printf("<ul>%n");  
                out.printf("<li>error code: %s</li>%n", m.getErrorCode());  
                out.printf("<li>message: %s</li>%n", m.getMessage());  
                out.printf("<li>details: %s</li>%n", m.getErrorDetails());  
                out.printf("</ul>%n");  
            } else {  
                out.printf("<p>%s</p>%n", m.getMessage());  
  
                out.printf("<table>%n");  
                out.printf("<tr>%n");  
                out.printf("<td>Badge</td><td>Surname</td><td>Age</td><td>Salary</td>%n");  
                out.printf("</tr>%n");  
  
                for (Employee e : el) {  
                    out.printf("<tr>%n");  
                    out.printf("<td>%s</td><td>%s</td><td>%s</td><td>%s</td>%n", e.getBadge(), e.getSurname(),  
                        e.getAge(), e.getSalary());  
                    out.printf("</tr>%n");  
                }  
                out.printf("</table>%n");  
  
                out.printf("</body>%n");  
                out.printf("</html>%n");  
  
                // flush the output stream buffer  
                out.flush();  
  
                // close the output stream  
                out.close();  
            } catch (IOException ex) {  
                LOGGER.error(new StringFormattedMessage("Unable to send response when creating employee %d.", salary), ex);  
                throw ex;  
            } finally {  
                LogContext.removeIPAddress();  
                LogContext.removeAction();  
                LogContext.removeUser();  
            }  
        }  
    }  
}
```

Writes out the HTML page

Questions?

WE HAD A MASSIVE DATA BREACH. HACKERS GOT INTO THE PRIVATE DATA OF ALL OF OUR CUSTOMERS.



DILBERT.COM @SCOTTADAMSSAYS

NO PROBLEM. WE'LL ISSUE A PRESS RELEASE THAT SAYS WE'RE SORRY AND IT WILL NEVER HAPPEN AGAIN.



5-7-18 ©2018 Scott Adams, Inc./Dist. by Andrews McMeel

THAT'S WHAT WE SAID THE LAST THREE TIMES IT HAPPENED.



OUR STRATEGY IS TO WEAR THEM DOWN.

